Case Study(Solution)
(Bynry Inc.)

Part 1: Code Review & Debugging (Issues Identification & Explanation)

1. Lack of Atomic Transactions

The implementation uses two separate commit() calls. This makes the operation non-atomic. If the second commit fails after the first succeeds, the database will contain a product without inventory, leading to inconsistent data. There is also no rollback mechanism.

2. No Input Validation

The API directly accesses request data without validation. Missing fields can cause KeyError, incorrect types may pass through, and malicious input could be injected.

3. No Authentication or Authorization

The endpoint is publicly accessible without any authentication or role checks. Any user can create products or flood the system.

4. No Duplicate SKU Check

The system does not verify if a SKU already exists. This can result in duplicate entries or database crashes if constraints exist.

5. No Validation of warehouse_id

The code assumes the provided warehouse exists. If it does not, it may either silently fail or crash.

6. No Error Handling

There is no try-except block. Any exception results in a raw server error and may leave the database in an inconsistent state.

7. Price Validation Missing

The price field is not validated. Negative or zero values are allowed, which is logically incorrect for most systems.

8. Initial Quantity Not Validated

The quantity can be negative, missing, or of incorrect type.

9. request.json May Be None

If the request body is invalid or missing, request.json returns None, causing runtime errors.

10. Incorrect Business Logic for warehouse_id

The same warehouse_id is used for both Product and Inventory. This design limits flexibility and assumes incorrect relationships.

11. Incorrect HTTP Status Code

The API returns 200 OK instead of 201 Created after successful resource creation.

12. No Rate Limiting

There is no restriction on request frequency, making the system vulnerable to abuse or denial-of-service attacks.

13. Fragile Access to product.id

After committing, accessing product.id can be unreliable due to session behavior.

14. No Logging

There is no logging of operations, errors, or user actions.

15. Not Idempotent

Repeated requests can create duplicate records.

16. Incomplete Response Body

The API only returns product_id, requiring additional calls to fetch details.

17. No Audit Trail

There is no tracking of who created the product or when.

18. No Explicit Schema Definition

The API relies on implicit field access without documentation or schema definition.

19. No Content-Type Enforcement

The API does not enforce application/json, leading to unclear errors.

20. Mixed Responsibilities

The endpoint combines product creation and inventory initialization, which should ideally be separate operations.

21. No Testing

There are no unit or integration tests covering key scenarios such as validation failures or database errors.

Part 1: Code Review & Debugging (Code Fix and Explanation)

1. Code Fix Explanation

The original code was a Flask API endpoint to create a product and add it to inventory, but it had serious gaps. There was no input validation, so missing or invalid fields like a negative price would go straight into the database without any checks. The two separate `commit()` calls meant if the second one failed, the product would be saved with no inventory — a broken state. No error handling meant any failure returned a raw crash to the caller. Authentication was missing entirely, so anyone could call the endpoint. The fixes added schema validation to catch bad input early, combined both database writes into one transaction so they either both succeed or both fail, wrapped everything in proper error handling with a rollback on failure, added a role-based auth check, and corrected the response to return `201 Created` with full product details.

Assumptions Made for corrected code

Since the original code was just a single function with no surrounding context, these are the assumptions made to write the corrected version:

About the models
- A `Warehouse` model exists in the database and can be queried to verify a warehouse before assigning products to it
- The `Product` model has a `created_by` column and a `created_at` column for audit tracking
- The `Inventory` model accepts a `product_id`, `warehouse_id`, and `quantity` — same as the original implied

About the auth system
- A `User` model exists with a `role` field (like `admin` or `inventory_manager`)
- JWT-based authentication is either already in place or acceptable as the auth mechanism
- The calling client sends a valid JWT token in the `Authorization` header

About the database setup
- Foreign key constraints exist between `Inventory` and `Product`, and between both and `Warehouse` — which is why the warehouse check matters
- SQLAlchemy is configured with standard defaults, including `expire_on_commit=True`, which is why `flush()` was used instead of relying on `product.id` after commit

About the business rules
- A SKU is meant to be unique across all products — the original code did not enforce this but the intent was clearly there
- A price of zero or below is not valid for this business
- Negative inventory quantity is not physically meaningful and should be rejected

About the environment
- Logging is configured at the application level so `logging.getLogger(__name__)` will work
- The application uses Flask with SQLAlchemy and marshmallow, which were already implied by the original code's style

2. **Fixed Code**

```
import logging
from functools import wraps

from flask import Flask, request, jsonify, g
from flask_jwt_extended import JWTManager, verify_jwt_in_request,
get_jwt_identity
from flask_sqlalchemy import SQLAlchemy
from marshmallow import Schema, fields, validate, ValidationError
from sqlalchemy.exc import IntegrityError
from datetime import datetime

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///inventory.db'
app.config['JWT_SECRET_KEY'] = 'change-this-in-production'

db = SQLAlchemy(app)
jwt = JWTManager(app)
logger = logging.getLogger(__name__)


class User(db.Model):
    _tablename_= 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(100), unique=True, nullable=False)
    role = db.Column(db.String(50), nullable=False)


class Warehouse(db.Model):
    _tablename_ = 'warehouses'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(255), nullable=False)
```

```
class Product(db.Model):
    _tablename_ = 'products'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(255), nullable=False)
    sku = db.Column(db.String(100), unique=True, nullable=False)
    price = db.Column(db.Float, nullable=False)
    warehouse_id = db.Column(db.Integer, db.ForeignKey('warehouses.id'),
nullable=False)
    created_by = db.Column(db.Integer, db.ForeignKey('users.id'), nullable=False)
    created_at = db.Column(db.DateTime, default=datetime.utcnow)


class Inventory(db.Model):
    _tablename_ = 'inventory'

    id = db.Column(db.Integer, primary_key=True)
    product_id = db.Column(db.Integer, db.ForeignKey('products.id'),
nullable=False)
    warehouse_id = db.Column(db.Integer, db.ForeignKey('warehouses.id'),
nullable=False)
    quantity = db.Column(db.Integer, nullable=False)


class CreateProductSchema(Schema):
    name = fields.Str(required=True, validate=validate.Length(min=1, max=255))
    sku = fields.Str(
        required=True,
        validate=[
            validate.Length(min=1, max=100),
            validate.Regexp(
                r'^[A-Za-z0-9\-_]+$',
                error='SKU may only contain letters, numbers, hyphens, and
underscores'
            )
        ]
    )
```

```
    price = fields.Float(required=True, validate=validate.Range(min=0.01,
error='Price must be greater than zero'))
    warehouse_id = fields.Int(required=True)
    initial_quantity = fields.Int(required=True,
validate=validate.Range(min=0, error='Quantity cannot be negative'))

_create_product_schema = CreateProductSchema()

def require_role(*roles):
 def decorator(fn):
  @wraps(fn)
   def wrapper(*args, **kwargs):
        verify_jwt_in_request()
        user_id = get_jwt_identity()
        user = User.query.get(user_id)

        if user is None:
            return jsonify({'error': 'User not found'}), 401

        if user.role not in roles:
            return jsonify({'error': 'You do not have permission to perform
 this action'}), 403

        g.current_user = user
        return fn(*args, **kwargs)
    return wrapper
   return decorator
@app.route('/api/products', methods=['POST'])
@require_role('admin', 'inventory_manager')
def create_product():
  if not request.is_json:
   return jsonify({'error': 'Content-Type must be application/json'}), 415

   raw = request.get_json(silent=True)
   if raw is None:
      return jsonify({'error': 'Request body is missing or contains malformed
JSON'}), 400
```

```
try:
  data = _create_product_schema.load(raw)
 except ValidationError as err:
  return jsonify({'error': 'Validation failed', 'details': err.messages}), 400

 existing = Product.query.filter_by(sku=data['sku']).first()
 if existing:
  return jsonify({'error': 'A product with this SKU already exists'}), 409

 warehouse = Warehouse.query.get(data['warehouse_id'])
 if warehouse is None:
  return jsonify({'error': 'Warehouse not found'}), 404

try:
  product = Product(
    name=data['name'],
    sku=data['sku'],
    price=data['price'],
    warehouse_id=data['warehouse_id'],
  created_by=g.current_user.id
)
db.session.add(product)
db.session.flush()

inventory = Inventory(
  product_id=product.id,
  warehouse_id=data['warehouse_id'],
  quantity=data['initial_quantity']
)
db.session.add(inventory)
db.session.commit()
```

```
except IntegrityError as exc:
    db.session.rollback()
    logger.error(
      'IntegrityError on product creation: sku=%s user=%s error=%s',
      data.get('sku'), g.current_user.id, str(exc)
    )
    return jsonify({'error': 'A database constraint was violated. Check that the SKU and warehouse are valid.'}), 409

  except Exception:
    db.session.rollback()
    logger.exception(
      'Unexpected error on product creation: sku=%s user=%s',
      data.get('sku'), g.current_user.id
    )
    return jsonify({'error': 'An unexpected error occurred. Please try again later.'}), 500

  logger.info(
    'Product created: id=%s sku=%s warehouse_id=%s user=%s',
    product.id, product.sku, product.warehouse_id, g.current_user.id
  )

  return jsonify({
    'message': 'Product created successfully',
    'product': {
      'id': product.id,
      'name': product.name,
      'sku': product.sku,
      'price': product.price,
      'warehouse_id': product.warehouse_id,
      'initial_quantity': inventory.quantity,
      'created_by': product.created_by,
      'created_at': product.created_at.isoformat()
    }
  }), 201
```

```
if __name__ == '__main__':
  with app.app_context():
    db.create_all()
  app.run(debug=False)
```

Part 2: Database Design

Design Schema:

#Companies

```
CREATE TABLE companies (
 id                SERIAL          PRIMARY KEY,
 name              VARCHAR(255)    NOT NULL,
 legal_name        VARCHAR(255),
 tax_id            VARCHAR(100)    UNIQUE,
 address_line1     VARCHAR(255),
 address_line2     VARCHAR(255),
 city              VARCHAR(100),
 state             VARCHAR(100),
 country           VARCHAR(100)    NOT NULL DEFAULT 'US',
 postal_code       VARCHAR(20),
 email             VARCHAR(255),
 phone             VARCHAR(50),
 is_active         BOOLEAN         NOT NULL DEFAULT TRUE,
 created_at        TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
 updated_at        TIMESTAMPTZ     NOT NULL DEFAULT NOW()
);
```

#Warehouses

```
CREATE TABLE warehouses (
 id                SERIAL          PRIMARY KEY,
 company_id        INT             NOT NULL REFERENCES
                                   companies(id) ON DELETE
RESTRICT,
 name              VARCHAR(255)    NOT NULL,
 code              VARCHAR(50)     NOT NULL,
```

```
address_line1      VARCHAR(255),
 city              VARCHAR(100),
 state             VARCHAR(100),
 country           VARCHAR(100)     NOT NULL DEFAULT 'US',
 postal_code       VARCHAR(20),
 max_capacity      NUMERIC(15, 2),
 is_active         BOOLEAN          NOT NULL DEFAULT TRUE,
 created_at        TIMESTAMPTZ      NOT NULL DEFAULT NOW(),
 updated_at        TIMESTAMPTZ      NOT NULL DEFAULT NOW(),

 CONSTRAINT uq_warehouse_code_per_company UNIQUE (company_id,
code)
);
CREATE INDEX idx_warehouses_company ON warehouses(company_id);
```

#Suppliers

```
CREATE TABLE suppliers (
 id                SERIAL           PRIMARY KEY,
 name              VARCHAR(255)     NOT NULL,
 contact_name      VARCHAR(255),
 email             VARCHAR(255),
 phone             VARCHAR(50),
 address_line1     VARCHAR(255),
 city              VARCHAR(100),
 country           VARCHAR(100),
 payment_terms     VARCHAR(100),
 is_active         BOOLEAN          NOT NULL DEFAULT TRUE,
 created_at        TIMESTAMPTZ      NOT NULL DEFAULT NOW(),
 updated_at        TIMESTAMPTZ      NOT NULL DEFAULT NOW()
);
```

#Company <-> Supplier (many - many)

```
CREATE TABLE company_suppliers (
 id                    SERIAL                    PRIMARY KEY,
 company_id            INT                       NOT NULL REFERENCES companies(id)
                                                 ON DELETE

CASCADE,
 supplier_id           INT                       NOT NULL REFERENCES suppliers(id)
                                                 ON DELETE
CASCADE,
 account_
number                 VARCHAR(100),
is_preferred           BOOLEAN                   NOT NULL DEFAULT FALSE,
 contracted_
since                  DATE,
 created_at            TIMESTAMPTZ               NOT NULL DEFAULT NOW(),

 CONSTRAINT uq_company_supplier UNIQUE (company_id, supplier_id)
);

CREATE INDEX idx_company_suppliers_company ON
company_suppliers(company_id);
CREATE INDEX idx_company_suppliers_supplier ON
company_suppliers(supplier_id);
```

#Products

```
CREATE TABLE products (
 id                    SERIAL                    PRIMARY KEY,
 sku                   VARCHAR(100)              NOT NULL UNIQUE,
 name                  VARCHAR(255)              NOT NULL,
 description           TEXT,
 unit_of_
measure                VARCHAR(50)               NOT NULL DEFAULT 'each',
 weight_kg             NUMERIC(10, 4),
 dimensions_cm         JSONB,
```

```
category          VARCHAR(100),
is_bundle         BOOLEAN          NOT NULL DEFAULT FALSE,
is_active         BOOLEAN          NOT NULL DEFAULT TRUE,
 created_at       TIMESTAMPTZ      NOT NULL DEFAULT NOW(),
 updated_at       TIMESTAMPTZ      NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_products_sku ON products(sku);
CREATE INDEX idx_products_is_bundle ON products(is_bundle)
WHERE is_bundle = TRUE;
CREATE INDEX idx_products_category ON products(category);
```

#Product Bundles

```
CREATE TABLE product_bundles (
 id                  SERIAL          PRIMARY KEY,
 parent_product_id   INT             NOT NULL REFERENCES products(id) ON
                                                DELETE


CASCADE,
 child_product_id    INT             NOT NULL REFERENCES products(id) ON
                                                DELETE
 RESTRICT,
 quantity            NUMERIC(10, 4)  NOT NULL CHECK (quantity > 0),
 created_at          TIMESTAMPTZ     NOT NULL DEFAULT NOW()

    CONSTRAINT uq_bundle_component UNIQUE (parent_product_id,
child_product_id),
    CONSTRAINT chk_no_self_bundle CHECK (parent_product_id <>
child_product_id)
);

CREATE INDEX idx_product_bundles_parent ON
product_bundles(parent_product_id);
CREATE INDEX idx_product_bundles_child ON
product_bundles(child_product_id);
```

#Supplier Products

```
CREATE TABLE supplier_products (
 id                SERIAL                PRIMARY KEY,
 supplier_id        INT        NOT NULL REFERENCES suppliers(id) ON
                                                DELETE
CASCADE,
 product_id         INT        NOT NULL REFERENCES products(id) ON
                                                DELETE
CASCADE,
supplier_sku    VARCHAR(100),

 unit_cost        NUMERIC(12, 4)     NOT NULL CHECK (unit_cost >= 0),
 currency            CHAR(3)              NOT NULL DEFAULT 'USD',
 lead_time_days      INT                 CHECK (lead_time_days >= 0),
 minimum_
order_qty        NUMERIC(10, 4)        CHECK (minimum_order_qty > 0),
 is_preferred      BOOLEAN              NOT NULL DEFAULT FALSE,

effective_
from              DATE NOT NULL          DEFAULT CURRENT_DATE,
 effective_to         DATE,

 created_at     TIMESTAMPTZ              NOT NULL DEFAULT NOW(),
 updated_at     TIMESTAMPTZ              NOT NULL DEFAULT NOW(),

    CONSTRAINT uq_supplier_product_period UNIQUE (supplier_id,
product_id, effective_from),
    CONSTRAINT chk_effective_dates CHECK (effective_to IS NULL OR
effective_to > effective_from)
);

CREATE INDEX idx_supplier_products_supplier ON
supplier_products(supplier_id);
CREATE INDEX idx_supplier_products_product ON
supplier_products(product_id);
```

#Warehouse Inventory

```
CREATE TABLE warehouse_inventory (
 id                    SERIAL                PRIMARY KEY,
 warehouse_id          INT          NOT NULL REFERENCES warehouses(id)
                                          ON DELETE RESTRICT,
 product_id            INT          NOT NULL REFERENCES products(id) ON
                                                  DELETE
 RESTRICT,
 quantity_
on_hand          NUMERIC(15, 4)       NOT NULL DEFAULT 0 CHECK
                                        (quantity_on_hand >= 0),
  quantity_
reserved         NUMERIC(15, 4)       NOT NULL DEFAULT 0 CHECK
                                         (quantity_reserved >= 0),
last_counted
_at                TIMESTAMPTZ,

    CONSTRAINT uq_warehouse_product UNIQUE (warehouse_id,
product_id),
    CONSTRAINT chk_reserved_lte_onhand CHECK (quantity_reserved <=
quantity_on_hand)
);

CREATE INDEX idx_wh_inventory_warehouse ON
warehouse_inventory(warehouse_id);
CREATE INDEX idx_wh_inventory_product ON
warehouse_inventory(product_id);
CREATE INDEX idx_wh_inventory_reorder ON
warehouse_inventory(quantity_on_hand)
 WHERE reorder_point IS NOT NULL;
```

#Inventory Transactions

```
CREATE TYPE txn_type AS ENUM (
 'RECEIPT',
 'SALE',
 'TRANSFER_IN',
 'TRANSFER_OUT',
 'ADJUSTMENT_UP',
 'ADJUSTMENT_DOWN',
 'RETURN',
 'DAMAGE',
 'BUNDLE_ASSEMBLE',
 'BUNDLE_DISASSEMBLE'
);

CREATE TABLE inventory_transactions (
 id                    BIGSERIAL                    PRIMARY KEY,
 warehouse
_id                    INT                NOT NULL REFERENCES warehouses(id) ON
                                                    DELETE
RESTRICT,
 product_id       INT                NOT NULL REFERENCES products(id) ON
                                                    DELETE
RESTRICT,
 txn_type         txn_type                          NOT NULL,
 quantity_
delta             NUMERIC(15, 4)                    NOT NULL,
 quantity_
before            NUMERIC(15, 4)                    NOT NULL,
 quantity_
after             NUMERIC(15, 4)                    NOT NULL,
 unit_
cost              NUMERIC(12, 4),
 reference_
type              VARCHAR(50),
```

```
 reference_
id                    INT,
 notes                TEXT,
 performed_by    INT,
 performed_
 at                TIMESTAMPTZ                NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_inv_txn_warehouse_product ON
inventory_transactions(warehouse_id, product_id);
CREATE INDEX idx_inv_txn_performed_at ON
inventory_transactions(performed_at DESC);
CREATE INDEX idx_inv_txn_reference ON
inventory_transactions(reference_type, reference_id)
 WHERE reference_id IS NOT NULL;
CREATE INDEX idx_inv_txn_type ON inventory_transactions(txn_type);
```

Questions list for product team

1. Do companies sell to each other, or only to end customers?
2. Is there a concept of locations within a warehouse (zones, aisles, bins)?
3. Can bundles contain other bundles (nested bundles)? What is the max nesting depth?
4. Can a warehouse be owned by multiple companies (shared/3PL warehouses)?
5. Is there a concept of supplier contracts (volume discounts, tiered pricing)?
6. Is there a requirement to never delete inventory or product records (soft-delete only)?

Design Decisions & Justifications

1. **Append-Only Audit Log (inventory_transactions)**

The inventory_transactions table is designed as an append-only audit log to maintain a complete and reliable history of all inventory movements . Instead of updating or deleting records, new entries are added for every change.
The use of BIGSERIAL instead of SERIAL is important because this table is expected to grow very large over time. A standard integer would eventually overflow at around 2 billion records, whereas BIGSERIAL provides a much higher limit.
Fields like quantity_before and quantity_after are included to store the exact stock levels before and after each transaction. This allows easy auditing without needing to recalculate values from past data.
A single field, quantity_delta, is used to represent both incoming and outgoing stock. Positive values indicate additions, while negative values indicate reductions. This simplifies the structure and avoids unnecessary duplication.
The txn_type is implemented using an ENUM to restrict values to a fixed set of valid transaction types. This prevents incorrect or inconsistent entries due to manual errors.
The performed_by field does not use a foreign key because user management is likely handled by a separate system. This avoids tight coupling between systems.
Similarly, reference_type and reference_id are used instead of strict foreign keys to allow flexibility in linking transactions to different types of records, such as purchase orders or sales orders.

2. Dual-Layer Inventory Design

The system separates inventory data into two parts:

- warehouse_inventory for the current stock
- inventory_transactions for the historical record

This separation improves both performance and clarity.
For example, when checking current stock levels, queries are made on warehouse_inventory, which is optimized for fast lookups. On the other hand, when analyzing past activity or tracking changes, inventory_transactions is used.

To maintain consistency, updates to warehouse_inventory are performed in the same database transaction as inserting a new record into inventory_transactions. This ensures both tables always stay synchronized.

3. Product Bundles

The schema supports product bundles using a self-referential structure . A product marked with is_bundle = TRUE acts as a parent, and its components are stored in the product_bundles table.
A CHECK constraint ensures that a product cannot reference itself as a component. However, more complex cases like circular dependencies (for example, A includes B and B includes A) cannot be handled directly in SQL and must be validated in the application layer.
The is_bundle flag helps in quickly identifying bundle products without needing to join additional tables, which improves query performance.

4. Supplier Product Versioning (supplier_products)

The supplier_products table is designed to support price history instead of overwriting existing data.
The fields effective_from and effective_to define the validity period of a price. If effective_to is NULL, it means the price is currently active.
A unique constraint on (supplier_id, product_id, effective_from) ensures that duplicate pricing records with the same start date are not created, maintaining data consistency.

5. Reserved Inventory (quantity_reserved)

The warehouse_inventory table includes a quantity_reserved field to track stock that has been allocated but not yet shipped.
This allows the system to distinguish between total stock and available stock. The available quantity can be calculated as:
Available = quantity_on_hand − quantity_reserved
A CHECK constraint ensures that the reserved quantity never exceeds the actual stock, preventing invalid data.

6. Indexing Strategy

Indexes are added based on common query patterns to improve performance.

- idx_warehouses_company helps quickly find all warehouses belonging to a company.
- idx_wh_inventory_warehouse and idx_wh_inventory_product support fast inventory lookups.
- A partial index idx_wh_inventory_reorder is used to efficiently identify items that need restocking, without scanning the entire table.
- The composite index idx_inv_txn_warehouse_product improves performance for queries filtering by both warehouse and product.
- idx_inv_txn_performed_at DESC helps retrieve recent transactions quickly.
- Another partial index on transaction references improves lookup performance while ignoring NULL values.

Overall, indexing is carefully designed to balance query speed and storage efficiency.

7. Use of NUMERIC Data Type

The schema uses NUMERIC for quantities and monetary values instead of floating-point types.
This is because floating-point numbers can introduce rounding errors due to their internal representation. In contrast, NUMERIC provides exact precision, which is critical for financial calculations and inventory accuracy.

8. Use of TIMESTAMPTZ

All timestamps are stored using TIMESTAMPTZ, which includes timezone information.
This ensures that time-related data remains consistent across different regions and avoids ambiguity in systems that may operate in multiple time zones.

9. **Conclusion**

Overall, the design focuses on maintaining data integrity, ensuring high performance, and supporting scalability. Constraints and ENUMs enforce correctness, indexes optimize queries, and the audit log structure ensures full traceability. At the same time, flexibility is maintained by avoiding unnecessary tight coupling and allowing certain complex rules to be handled at the application level.

3. **API Implementation**

Implementation

```
import logging
from datetime import datetime, timedelta, timezone
from functools import wraps

from flask import Flask, jsonify, g
from flask_jwt_extended import JWTManager, verify_jwt_in_request,
get_jwt_identity
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import func, and_
from sqlalchemy.exc import SQLAlchemyError

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///inventory.db'
app.config['JWT_SECRET_KEY'] = 'change-this-in-production'

db = SQLAlchemy(app)
jwt = JWTManager(app)
logger = logging.getLogger(__name__)

RECENT_SALE_DAYS = 30
MAX_DAYS_STOCKOUT = 999
```

```
# Fallback thresholds when warehouse_inventory.reorder_point is NULL
CATEGORY_THRESHOLDS = {
  'electronics': 50,
  'perishable': 100,
  'raw_material': 200,
  'consumable': 75,
  'spare_parts': 30,
}
DEFAULT_THRESHOLD = 10


class Company(db.Model):
      _tablename_ = 'companies'
       id    = db.Column(db.Integer, primary_key=True)
       name   = db.Column(db.String(255), nullable=False)
       is_active = db.Column(db.Boolean, nullable=False, default=True)



class Warehouse(db.Model):
      tablename  = 'warehouses'
      id    = db.Column(db.Integer, primary_key=True)
      company_id = db.Column(db.Integer, db.ForeignKey('companies.id'),
nullable=False)
      name   = db.Column(db.String(255), nullable=False)
      is_active = db.Column(db.Boolean, nullable=False, default=True)



class Product(db.Model):
       _tablename_ = 'products'
       id    = db.Column(db.Integer, primary_key=True)
       name   = db.Column(db.String(255), nullable=False)
       sku   = db.Column(db.String(100), unique=True, nullable=False)
       category = db.Column(db.String(100))
       is_active = db.Column(db.Boolean, nullable=False, default=True)
```

```
class WarehouseInventory(db.Model):
    _tablename_= 'warehouse_inventory'
    id        = db.Column(db.Integer, primary_key=True)
    warehouse_id   = db.Column(db.Integer,
db.ForeignKey('warehouses.id'), nullable=False)
    product_id    = db.Column(db.Integer, db.ForeignKey('products.id'),
nullable=False)
    quantity_on_hand = db.Column(db.Numeric(15, 4), nullable=False,
default=0)
    quantity_reserved = db.Column(db.Numeric(15, 4), nullable=False,
default=0)
    reorder_point  = db.Column(db.Numeric(15, 4)) # NULL = use
category/global default


class InventoryTransaction(db.Model):
    _tablename_ = 'inventory_transactions'
    id      = db.Column(db.BigInteger, primary_key=True)
    warehouse_id = db.Column(db.Integer,
db.ForeignKey('warehouses.id'), nullable=False)
    product_id  = db.Column(db.Integer, db.ForeignKey('products.id'),
nullable=False)
    txn_type   = db.Column(db.String(50), nullable=False)
    quantity_delta = db.Column(db.Numeric(15, 4), nullable=False) #
negative = outbound
    performed_at = db.Column(db.DateTime(timezone=True),
nullable=False, default=datetime.utcnow)


class Supplier(db.Model):
    _tablename_ = 'suppliers'
    id    = db.Column(db.Integer, primary_key=True)
    name   = db.Column(db.String(255), nullable=False)
    email  = db.Column(db.String(255))
    is_active = db.Column(db.Boolean, nullable=False, default=True)
```

```
class CompanySupplier(db.Model):
  _tablename_ = 'company_suppliers'
    id     = db.Column(db.Integer, primary_key=True)
    company_id = db.Column(db.Integer, db.ForeignKey('companies.id'),
nullable=False)
     supplier_id = db.Column(db.Integer, db.ForeignKey('suppliers.id'),
nullable=False)


class SupplierProduct(db.Model):
        _tablename_ = 'supplier_products'
        id      = db.Column(db.Integer, primary_key=True)
        supplier_id  = db.Column(db.Integer, db.ForeignKey('suppliers.id'),
nullable=False)
        product_id  = db.Column(db.Integer, db.ForeignKey('products.id'),
nullable=False)
        is_preferred = db.Column(db.Boolean, nullable=False, default=False)
        effective_from = db.Column(db.Date, nullable=False)
        effective_to = db.Column(db.Date) # NULL = currently active


class User(db.Model):
        _tablename_ = 'users'
        id   = db.Column(db.Integer, primary_key=True)
        username = db.Column(db.String(100), unique=True, nullable=False)
        role  = db.Column(db.String(50), nullable=False)
```

```
def require_role(*roles):
  def decorator(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
      verify_jwt_in_request()
      user = User.query.get(get_jwt_identity())
      if user is None:
        return jsonify({'error': 'User not found'}), 401
      if user.role not in roles:
        return jsonify({'error': 'Forbidden'}), 403
      g.current_user = user
      return fn(*args, **kwargs)
    return wrapper
  return decorator
def _resolve_threshold(inv: WarehouseInventory, product: Product) ->
float:
  """Per-row reorder_point → category default → global default."""
  if inv.reorder_point is not None:
    return float(inv.reorder_point)
  return float(CATEGORY_THRESHOLDS.get((product.category or
'').lower(), DEFAULT_THRESHOLD))


def _avg_daily_sales(warehouse_id: int, product_id: int, since: datetime) ->
float:
  """Average units sold per day over the recent window."""
  total = db.session.query(

func.coalesce(func.sum(func.abs(InventoryTransaction.quantity_delta)),
0)
  ).filter(
    InventoryTransaction.warehouse_id == warehouse_id,
    InventoryTransaction.product_id == product_id,
    InventoryTransaction.txn_type  == 'SALE',
    InventoryTransaction.performed_at >= since,
  ).scalar()
  return float(total) / RECENT_SALE_DAYS
```

```
def _days_until_stockout(available: float, daily_rate: float):
  """Returns int days or None if rate is zero (unknown velocity)."""
  if daily_rate <= 0:
    return None
  return min(round(available / daily_rate), MAX_DAYS_STOCKOUT)


def _get_supplier(product_id: int, company_id: int):
  """Returns the preferred active supplier for the product, scoped to the
company."""
  company_supplier_ids = (
    db.session.query(CompanySupplier.supplier_id)
    .filter(CompanySupplier.company_id == company_id)
    .subquery()
  )
  base = (
    db.session.query(Supplier)
    .join(SupplierProduct, SupplierProduct.supplier_id == Supplier.id)
    .filter(
      SupplierProduct.product_id == product_id,
      SupplierProduct.effective_to.is_(None),
      Supplier.is_active == True,
      Supplier.id.in_(company_supplier_ids),
    )
  )
  return base.filter(SupplierProduct.is_preferred == True).first() or
base.first()
@app.route('/api/companies/<int:company_id>/alerts/low-stock',
methods=['GET'])
@require_role('admin', 'inventory_manager', 'viewer')
def get_low_stock_alerts(company_id: int):
  company = Company.query.filter_by(id=company_id,
is_active=True).first()
  if company is None:
    return jsonify({'error': f'Company {company_id} not found or
inactive'}), 404
```

```
# Sub-select: only (warehouse, product) pairs with at least one recent sale
 recent_sales = (
  db.session.query(
    InventoryTransaction.warehouse_id,
    InventoryTransaction.product_id,
  )
  .filter(
    InventoryTransaction.warehouse_id.in_(warehouse_ids),
    InventoryTransaction.txn_type  == 'SALE',
    InventoryTransaction.performed_at >= sales_since,
  )
  .distinct()
  .subquery()
)

try:
  rows = (
    db.session.query(WarehouseInventory, Product)
    .join(Product, Product.id == WarehouseInventory.product_id)
    .join(
      recent_sales,
      and_(
        recent_sales.c.warehouse_id == WarehouseInventory.warehouse_id,
        recent_sales.c.product_id == WarehouseInventory.product_id,
      )
    )
    .filter(
      WarehouseInventory.warehouse_id.in_(warehouse_ids),
        Product.is_active == True,
    )
    .all()
  )
except SQLAlchemyError:
  logger.exception('low_stock_alerts DB error company_id=%s', company_id)
  return jsonify({'error': 'Database error. Please try again later.'}), 503
```

```
alerts = []
 for inv, product in rows:
 available = float(inv.quantity_on_hand) - float(inv.quantity_reserved)
 threshold = _resolve_threshold(inv, product)
   if available >= threshold:
     continue

   try:
     daily_rate = _avg_daily_sales(inv.warehouse_id, product.id,
sales_since)
   except SQLAlchemyError:
     logger.exception('low_stock_alerts: sales velocity error
product_id=%s', product.id)
     daily_rate = 0

   try:
     supplier = _get_supplier(product.id, company_id)
   except SQLAlchemyError:
     logger.exception('low_stock_alerts: supplier lookup error
product_id=%s', product.id)
     supplier = None

   alerts.append({
     'product_id':    product.id,
     'product_name':   product.name,
     'sku':       product.sku,
     'warehouse_id':   inv.warehouse_id,
     'warehouse_name':  warehouse_by_id[inv.warehouse_id].name,
     'current_stock':  round(available, 4),
     'threshold':    round(threshold, 4),
     'days_until_stockout': _days_until_stockout(available, daily_rate),
     'supplier': {
       'id':     supplier.id,
       'name':    supplier.name,
       'contact_email': supplier.email,
     } if supplier else None,
   })
```

```
if __name__ == '__main__':
  with app.app_context():
    db.create_all()
  app.run(debug=False)
```

Approach Overview

- The system is built using Flask with SQLAlchemy ORM, providing a structured and modular backend for handling inventory operations .
- JWT-based authentication and role-based access control are implemented to restrict API usage based on user roles (e.g., admin, inventory_manager).
- The API follows a service-oriented design, where helper functions handle specific logic such as threshold calculation, sales rate estimation, and supplier selection.
- A low-stock alert system is implemented by combining inventory data with recent sales transactions to identify items below threshold.
- The system calculates average daily sales dynamically over a fixed time window (e.g., last 30 days) to estimate demand trends.
- Derived metrics like available stock and days until stockout are computed in real time instead of being stored, ensuring up-to-date insights.
- Database queries are optimized using filtering, joins, and subqueries (e.g., selecting only products with recent sales) to reduce unnecessary data processing.
- Error handling and logging are included to ensure reliability and easier debugging in case of database or runtime failures.
- The design maintains loose coupling with suppliers and users, allowing flexibility and easier integration with external systems.